

|command process contract

|Заметка сделана 16 февр. 2026

|Процессы:

|Git flow:

|Основные ветки

1. `main` – релизная ветка, содержит код, который уже готов к выпуску. Каждый коммит здесь – это новый релиз.
 2. `def` – спринтовая ветка, результат одного спринта. Содержит стабильный код после завершения спринта.
 3. `feature` – ветка для разработки новых фич, каждая отдельная фича создаётся в своей атомарной ветке.
-

|Разработка фич

- Каждая фича создаётся как отдельная ветка от `future`, с префиксом `feat`.
 - Пример фичи: регистрация – отдельная ветка на эндпоинты, модель, тесты.
 - После завершения фичи её сквошем вливаем в ветку `feature`.
-

|Слияния

1. Малые фичи → сквош → `feature`
 2. В конце спринта → сквош → `def`
 3. После нескольких спринтов → сквош → `main`, формируя новый релиз
-

💡 Ключевая идея:

- `feat/*` → `feature` → `def` → `main`
- Все фичи атомарные, ветки небольшие, слияния сквошем для чистой истории.

Все мерджи - с флагом `squash`

|Writing code rules:

|1. Именованние

- **Функции:** имена должны быть **глаголом в инфинитиве**, без герундия (т.е. `get_user`, `create_order`).
 - **Переменные:** использовать **snake_case** (`user_name`, `order_list`).
 - **Константы:** **ALL_CAPS** (`MAX_RETRIES`, `DEFAULT_TIMEOUT`).
 - **Классы:** **PascalCase** (`UserProfile`, `OrderManager`).
-

|2. Типизация

- Всегда использовать **аннотации типов** для аргументов и возвращаемых значений, даже если Python динамический.
- Примеры:

```
def get_user_name(user_id: int) -> str:
```

```
...
```

|3. Тесты (Unit)

- Писать **unit-тесты** для всех функций, даже простых.
 - Исключение: функции-фабрики или очень тривиальные геттеры, но лучше всё равно тестировать.
-

|4. Документация

- Каждая функция **желательно** имеет краткую документацию (docstring), чтобы другие разработчики понимали её назначение.